

Decentralized Autonomous Architecture Oriented Calculator

Universidad Panamericana

Roberto Requejo Fernández 0236851

Abstract - This document has the purpose of presenting the development of a calculator with the implementation of a Decentralized Autonomous Architecture Oriented to the Services (ADSOA), where considerations from previous architectures such as Service autonomous decentralized (ADS) and Service Oriented Architecture (SOA) and its application in the JAVA programming language. Likewise, the importance of distributed computing is described, the necessary change to this computing model and the proposals that have occurred in the area.

1. Introduction

The rapid advancement of technology has led to the exploration of innovative approaches to computing and system implementation. This exploration has focused on not only enhancing resilience to failures but also on fostering interconnectivity. To address these needs, distributed computing has emerged as a pivotal solution. Distributed computing systems comprise multiple software components spread across different computers but function as a unified system.

Distributed computing aims to decentralize processes and resources, encompassing not only physical hardware elements but also delivering constant access and scalability. This approach is increasingly relevant in today's interconnected world, where many applications operate seamlessly without centralized servers, making server failures inconspicuous and system performance improvements achievable through distributed implementations.

2. Problem

The problem at hand lies in the development of a calculator that implements an Autonomous Decentralized Service-Oriented

Architecture (ADSOA). While it may seem like an excessive implementation due to the use of a server to process the operation.

3. Development

To solve this problem, we thought of making three programs. Described as it follows:

1. **Client:** This is a program that will load a friendly user interface capable of making basic operations such as addition, subtraction, division, and multiplication.
2. **Middleware:** This is message-oriented middleware (MOM) server that is uncharged of connecting everything and broadcast the packages to all clients and servers connected.
3. **Server:** The purpose of this program is to receive the packages and process de mathematical operations and send them back for the client to show to the user.

4. First Delivery

This first delivery consists of the three basic programs developed so that we get a basic idea of the program but not fully functional. For the communication between these three programs, we used the TCP protocol through the java sockets library, the communication between programs can be exemplified in the next diagram.

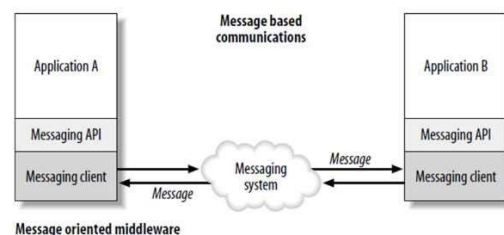


Fig 1. Application communication diagram

For the communication between the three programs, I developed a simple protocol for each program to do what is expected of them, since we broadcast all the packages the programs receive information, they don't use at all. The package I send contains for elements each separated by a coma, it looks

like this: “First Element, Second Element, Third Element, Forth Element, Fifth Element”. The first element contains whether a mathematical operation has been resolved or is pending to resolve, we use number codes. If the code is 100 means it is pending to resolve and if the code is 200 means it has been resolved. The second and third element is both operands, they are the numbers entered by the user. The fourth element is the type of mathematical operation, and it is represented by the symbol, the options are: +, -, * and /. And finally, the fifth element can be empty, because there we saved the result of the operation, so if the first element is 200, we know the fifth element is not going to be empty.

The middleware and the server have no graphic interface, but the client has, and it look like the following figure.



Fig 2. Calculator interface

Also here is a demonstration of my program functioning through images.

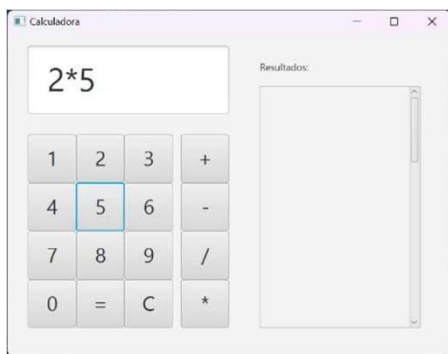


Fig 3. Example of user sending a request

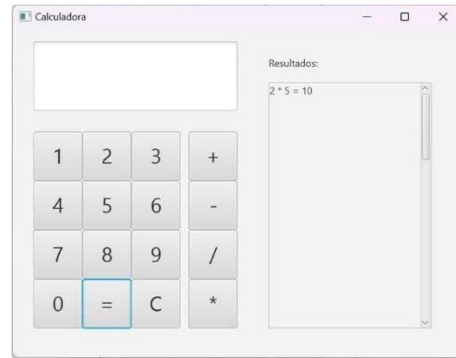


Fig 4. Example of response in the client

For this first delivery we can get a good idea of what is the goal of this project, but it has a lot of weak point until now. The main issue is that the middleware broadcast the message to everyone, and a lot of equal packages are generated, meaning that if we have more than one server open the client will receive more than one result and will display them all on the results bar. Also, if there is more than one client open it will display tall the result, even if it didn't request them. This is the main issue of this project as of now.

5. References

“¿Que es la arquitectura orientada a los servicios?”, ' Redhat.com. [En l´inea].

Disponible en:

<https://www.redhat.com/es/topics/cloud-nativeapps/what-is-service-oriented-architecture>. [Consultado: 14-Sep-2023]

M. Hinchey and L. Coyle, “Evolving critical systems: a research agenda for computer-based systems.”. University of Limerick, 01-Jan-2010 [En línea]. Disponible en:

<https://hdl.handle.net/10344/2085>. [Consultado: 10-Sep-2023]

“Broadcast”, Ecured.cu. [En línea]. Disponible en: <https://www.ecured.cu/Broadcast>.

[Consultado: 15-sept-2023].

IBM Cloud Education, “What is middleware?”, Ibm.com, 05-mar-2021. [En línea]. Disponible en:

<https://www.ibm.com/cloud/learn/middleware> e. [Consultado: 15- sept-2022]